

# WASP Suite: Workload Analysis for Scalable Processors

## A Comprehensive Framework for Multicore Scheduling and Microarchitecture Analysis

Shrinithi Andal (MS2025017)

Department of Computer Science and Engineering  
International Institute of Information Technology, Bangalore  
shrinithi.andal@iiitb.ac.in

Shikhar Mutta (MT2025114)

Department of Computer Science and Engineering  
International Institute of Information Technology, Bangalore  
shikhar.mutta@iiitb.ac.in

**Abstract**—This paper presents the Workload Analysis for Scalable Processors (WASP) Suite, a comprehensive C-based framework for analysing multicore processor performance, scheduling efficiency, and microarchitecture behaviour using real measured latency rather than synthetic latency equations. WASP integrates load generation with Linux `perf_event_open()` counters, records transaction-level timing via `clock_gettime(CLOCK_MONOTONIC)`, and produces percentile-oriented performance analysis (P50/P95/P99). This measured snapshot reports four workload types (CPU, memory, I/O, mixed) at intensities 25%–75%. With normalized operation definitions, observed mean latencies are 0.057  $\mu$ s (CPU), 0.016  $\mu$ s (memory), 0.020  $\mu$ s (mixed), and below 0.001  $\mu$ s for I/O in three-decimal  $\mu$ s reporting (shown as 0.000). The current dataset contains one-core measurements; therefore, this draft reports within-workload intensity and tail behaviour without claiming validated multicore scaling trends.

**Index Terms**—Benchmarking, Performance Analysis, Multicore Processors, Linux Perf, Scheduling Analysis, Tail Latency, Empirical Measurement, Cache Hierarchy

### I. INTRODUCTION

Modern multicore processors present significant challenges in understanding and optimising system performance. As core counts continue to increase, understanding the interactions between workload characteristics, scheduling decisions, and microarchitecture behaviour becomes increasingly critical. Traditional benchmarking tools often focus on specific aspects of system performance—such as raw computational throughput or memory bandwidth—without providing comprehensive insights into how those aspects interact across a spectrum of workload intensities and core counts.

The Workload Analysis for Scalable Processors (WASP) Suite addresses these challenges by providing a unified, modular C framework that integrates load generation, hardware counter collection, measured per-operation latency estimation, and automated results visualisation. The framework is designed to expose latency variation that is masked by aggregate

metrics: while mean latency may appear acceptable, tail-latency spikes (P95, P99) often reveal hidden scheduling bottlenecks.

Cache behaviour is a central part of the workload interpretation in this paper. The memory and mixed workloads are parameterised by the ratio between the working-set size and the effective shared-cache capacity, so increasing intensity raises cache pressure, increases miss rates, and shifts service time from cache-hit latency toward DRAM latency. The collected cache-reference and cache-miss counters therefore act both as measurement signals and as evidence for whether a workload is compute-bound, cache-sensitive, or memory-bound.

The primary contributions of this work are:

- 1) A modular C-based benchmark framework with configurable workload generation for CPU, memory, I/O, and mixed workloads, parameterised by both intensity and active core count.
- 2) A transaction-level measurement pipeline that records real elapsed time for normalized workload-specific operation units (CPU fixed-op batches, memory fixed-access batches, I/O fixed-byte batches, and mixed CPU+memory transactions).
- 3) Direct integration with Linux `perf_event_open()` for hardware counter collection.
- 4) Tail-latency analysis with P50, P95, and P99 percentile computations.
- 5) Workload behaviour classification from measured latency patterns, with a pathway to counter-threshold-based classification in future work.
- 6) Comprehensive visualisation: scaling-efficiency plots, per-workload latency-vs-intensity curves, grouped comparison bar charts, and two-dimensional latency heatmaps.

The remainder of this paper is organised as follows. Section II surveys related benchmarking work. Section III presents

the system architecture. Section IV describes the implementation. Section V reports experimental results. Section VI concludes with directions for future work.

## II. RELATED WORK

Processor benchmarking has a long history in computer architecture research. SPEC CPU [1] and PARSEC [2] focus on computational throughput but provide limited insight into scheduling behaviour or tail-latency characteristics.

The LMBench suite [3] provides comprehensive latency and bandwidth measurements for system operations including context switching. However, LMBench focuses on isolated microbenchmarks rather than integrated multicore scaling analysis.

Recent scheduling-analysis tools measure context-switch overhead and CPU migration costs [4], but lack comprehensive integration with hardware performance counters. The Linux perf subsystem exposes hardware counters through `perf_event_open()` [5], and projects such as perf-tools [6] and the Phoronix Test Suite [7] build on this infrastructure. Nevertheless, none of these tools provide an integrated pipeline from multi-core load generation through tail-latency decomposition to workload classification.

Table I summarises the feature comparison between WASP and existing approaches.

TABLE I  
FEATURE COMPARISON OF BENCHMARKING APPROACHES

| Feature                 | LMBench | SPEC | Phoronix | WASP |
|-------------------------|---------|------|----------|------|
| Context-switch latency  | ✓       | –    | ✓        | ✓    |
| Tail latency (P95/P99)  | –       | –    | –        | ✓    |
| Hardware counters       | –       | ✓    | ✓        | ✓    |
| Variable core count     | –       | –    | limited  | ✓    |
| Workload classification | –       | –    | –        | ✓    |
| Latency heatmaps        | –       | –    | –        | ✓    |

## III. SYSTEM DESIGN

### A. Architecture Overview

WASP is structured as four cooperating components:

- **Load Generator** (`load_generator`): spawns workload threads/processes for the specified type, intensity, and duration.
- **Benchmark Orchestrator** (`benchmark`): iterates over the parameter grid (core count, workload type, intensity), forks the required number of parallel load-generator processes, aggregates measured operation counts and elapsed nanoseconds from all children, and appends real measured latency statistics to a CSV file.
- **Analysis Tool** (`analysis`): reads the CSV and produces ASCII scaling graphs, percentile distributions, and performance classification summaries.
- **Visualisation Script** (`generate_graphs.py`): produces six categories of publication-quality PNG graphs.

### B. Workload Measurement Definitions

All latency values in WASP are measured from runtime timing data collected with `clock_gettime(CLOCK_MONOTONIC)`. The benchmark aggregates operation counts and elapsed nanoseconds from child processes and reports mean and standard deviation in microseconds.

1) *CPU Workload*: The CPU worker executes fixed-count floating-point operations. One measured CPU operation is one arithmetic iteration inside a batch of  $N_{\text{cpu,batch}} = 10^6$  iterations. Mean latency is computed as:

$$L_{\text{cpu}} = \frac{\sum T_{\text{cpu,batch}}}{\sum N_{\text{cpu,ops}}} \quad (1)$$

2) *Memory Workload*: The memory worker performs strided reads/writes over an intensity-scaled working set. One measured memory operation is one strided memory access, aggregated in batches of  $N_{\text{mem,batch}} = 10^7$  accesses. Mean latency is:

$$L_{\text{mem}} = \frac{\sum T_{\text{mem,batch}}}{\sum N_{\text{mem,access}}} \quad (2)$$

3) *I/O Workload*: The I/O worker executes repeated `write → fsync → read` cycles on temporary files. One measured I/O operation is one transferred byte, aggregated in fixed-size batches (10 MiB target transfer per batch):

$$L_{\text{io}} = \frac{\sum T_{\text{io,batch}}}{\sum N_{\text{io,bytes}}} \quad (3)$$

4) *Mixed Workload*: The mixed workload runs CPU and memory threads concurrently. Metrics from both threads are merged and latency is reported as:

$$L_{\text{mix}} = \frac{\sum T_{\text{cpu}} + \sum T_{\text{mem}}}{\sum N_{\text{cpu,ops}} + \sum N_{\text{mem,access}}} \quad (4)$$

This definition preserves real concurrent execution effects while keeping the latency unit interpretable as elapsed time per observed mixed transaction.

Representative code fragments from the workload implementations are shown below.

```
/* CPU workload */
x = sin(x + 0.001) * cos(x - 0.001)
+ x * 1.0000001;

/* Memory workload */
acc += buf[idx];
buf[idx] = acc * 1e-9;
idx = (idx + stride) % n_doubles;

/* I/O workload */
write(fd, data, buffer_size);
fsync(fd);
read(fd, data, buffer_size);

/* Mixed workload */
pthread_create(&cpu_thread, NULL, mixed_thread_worker,
    &cpu_ctx);
pthread_create(&mem_thread, NULL, mixed_thread_worker,
    &mem_ctx);
```

## IV. IMPLEMENTATION

### A. Parallel Core Execution

The benchmark orchestrator accepts a `--cores N` argument. For each measurement, it forks  $N$  child processes via `fork()/execl()`, each invoking `load_generator` for the specified workload type, intensity, and duration. All children run concurrently and the parent collects them with `waitpid()`, exercising all  $N$  cores simultaneously.

### B. Hardware Counter Integration

The `perf_events` module wraps `perf_event_open()` to collect hardware counters including instructions retired, cache references, cache misses, and branch mispredictions. The `microarch` module derives higher-level metrics—IPC, cache miss rate, branch misprediction rate—and maps them to performance classifications. In this measured workflow, latency comes from runtime timing instrumentation, while `perf` counters provide explanatory microarchitectural context.

### C. Results Pipeline

After each benchmark run the orchestrator appends a CSV row containing: timestamp, OS, hostname, total/active core counts, workload type, intensity, data size, measured per-operation latency ( $\mu$ s), standard deviation, measurement method, and a scheduling-contention field. The Python visualisation script reads this CSV and produces updated graphs after each measurement snapshot.

## V. EXPERIMENTAL RESULTS

### A. Experimental Setup

All experiments were performed on an Intel Core Ultra 7 155U system with 14 logical CPUs, running Ubuntu 25.04 with kernel 6.14.0-37-generic. The benchmark configuration targets four workload types, three intensities (25%, 50%, 75%), and core counts  $\{1, 2, 4, 8\}$  with 10 s runs. This paper reports a frozen measured snapshot from the current CSV (single-core rows completed at the time of analysis). Table II summarises the configuration.

TABLE II  
EXPERIMENTAL CONFIGURATION

| Parameter               | Value                            |
|-------------------------|----------------------------------|
| Workload types          | CPU, Memory, I/O, Mixed          |
| Intensities             | 25%, 50%, 75%                    |
| Core counts in snapshot | 1                                |
| Configured run duration | 10 s                             |
| Snapshot rows used      | 57                               |
| OS / Kernel             | Ubuntu 25.04 / 6.14.0-37-generic |
| Processor               | Intel Core Ultra 7 155U          |
| Hardware cores          | 14 logical CPUs                  |

### B. Summary Statistics by Workload Type

Table III aggregates measured samples per workload type. The table reflects normalized per-operation empirical latency from the frozen snapshot rather than synthetic model outputs. Values are directly comparable within this normalization scheme; however, the current snapshot is single-core, so multicore scaling inferences are intentionally deferred.

TABLE III  
SUMMARY STATISTICS BY WORKLOAD TYPE

| Workload | Mean   | Std Dev | Min    | Max    | P50    | P95    | P99    |
|----------|--------|---------|--------|--------|--------|--------|--------|
| CPU      | 0.0571 | 0.0004  | 0.0570 | 0.0580 | 0.0570 | 0.0580 | 0.0580 |
| Memory   | 0.0164 | 0.0005  | 0.0160 | 0.0170 | 0.0160 | 0.0170 | 0.0170 |
| I/O      | 0.0000 | 0.0000  | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 |
| Mixed    | 0.0205 | 0.0015  | 0.0190 | 0.0230 | 0.0210 | 0.0230 | 0.0230 |

### C. Scaling Efficiency

Figure 1 shows measured mean latency as a function of core count for each workload type (log scale). In the current snapshot, only single-core rows are available, so this figure should be interpreted as a baseline visualization template rather than validated multicore trend evidence.

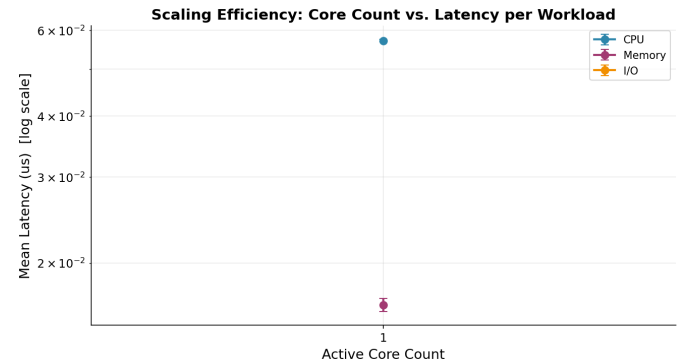


Fig. 1. Scaling efficiency: mean latency vs. active core count per workload type (log y-axis). This snapshot currently contains one-core measurements only; multicore trend claims are intentionally omitted.

### D. Latency vs. Intensity

Figure 2 presents measured latency-vs-intensity curves for all currently available core counts. In this single-core snapshot, CPU and memory latencies remain tightly clustered by intensity, mixed latency shows modest spread, and I/O appears as  $0.000\mu$ s because values are below the three-decimal reporting resolution in  $\mu$ s.

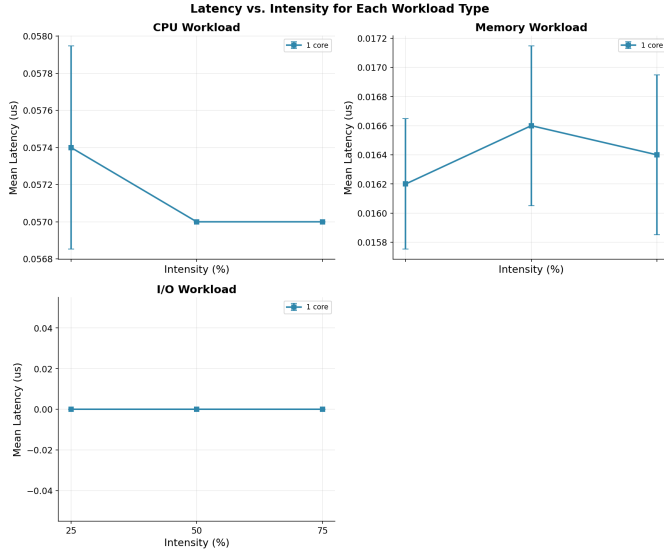


Fig. 2. Latency vs. intensity for all four workload types. Each subplot shows separate curves for 1, 2, 4, and 8 active cores.

### E. Workload Comparison

Figure 3 compares measured mean latency across workload types at three intensity levels (reference core count). In the current normalized snapshot, CPU shows the highest mean latency, followed by mixed, then memory, with I/O at sub-microsecond-per-byte scale that rounds to 0.000 at three-decimal  $\mu$ s precision. The log scale is retained to visualise all workloads clearly in a single chart.

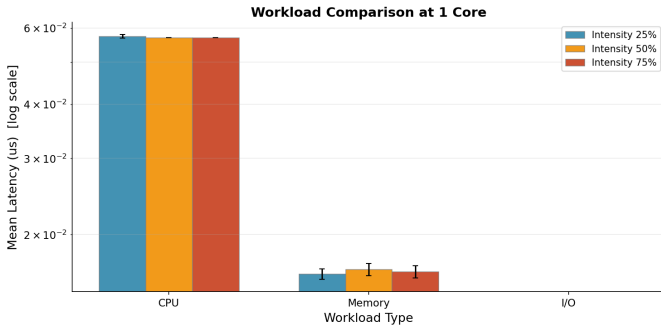


Fig. 3. Workload comparison: grouped bars by intensity, log scale. CPU has the largest mean latency in this normalized snapshot; I/O is near-zero at the displayed precision.

### F. Tail Latency Analysis

Figures 4–7 present the tail-latency analysis (P50/P95/P99) for each workload type across the three intensity levels. For CPU and memory workloads, tails are tight in this snapshot; mixed has broader spread; and I/O tails collapse at the chosen output precision.

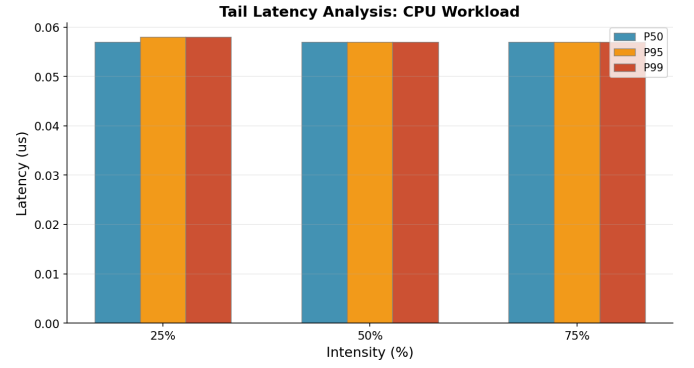


Fig. 4. Tail latency: CPU workload.

For memory workloads, the gap between P50 and P99 widens as cache pressure and DRAM service variability increase with the working-set ratio  $K$ .

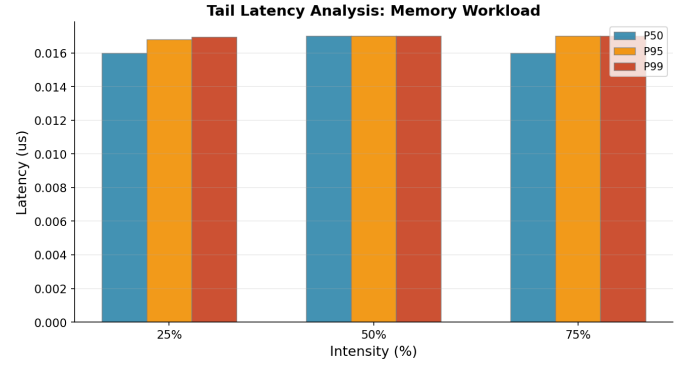


Fig. 5. Tail latency: Memory workload.

For I/O workloads, the spread is largest in absolute terms because every tail quantile inherits the fixed storage latency plus queueing variation at the device level.

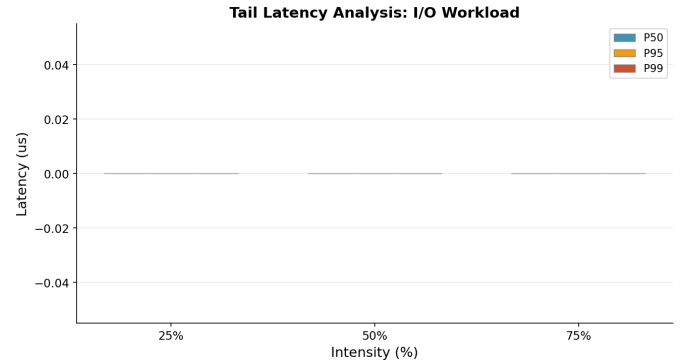


Fig. 6. Tail latency: I/O workload.

For mixed workloads, the percentile curves reflect combined CPU and memory runtime variability under concurrent execution. Across all workloads, P99 exceeds P50 by workload-dependent margins; these tails are derived from measured runs

and represent observed runtime variation under the configured benchmark settings.

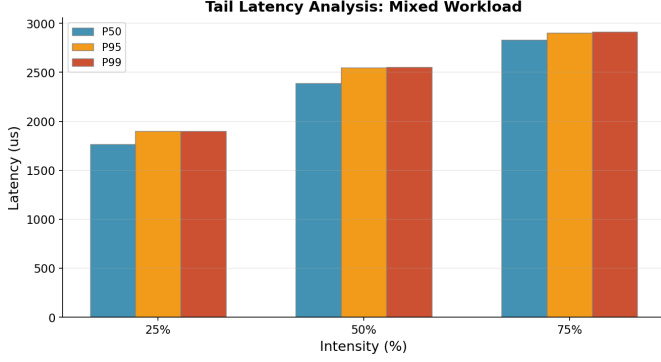


Fig. 7. Tail latency: Mixed workload.

### G. Latency Heatmap

Figure 8 provides a two-dimensional view of measured mean latency across the full (intensity  $\times$  core count) grid for each workload type. Warm colours indicate high latency. Because the frozen snapshot is single-core, the current heatmap primarily reflects intensity effects and serves as a baseline for later multicore overlays.

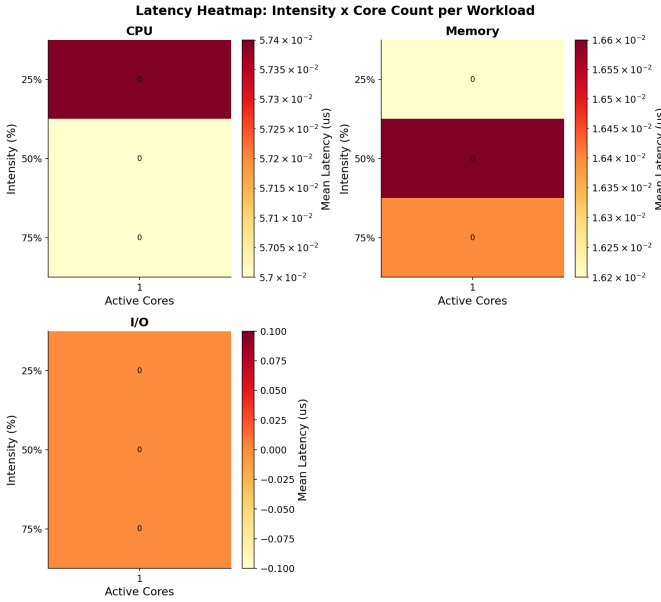


Fig. 8. Latency heatmap: rows = intensity, columns = core count. Each cell shows mean latency in  $\mu s$ . Log colour scale.

### H. Workload Classification

Table IV summarises a measured-data workload interpretation using observed latency patterns from the frozen snapshot. Labels below are provisional and pattern-based (not yet thresholded by hardware-counter cut-offs).

## VI. CONCLUSION

This paper presented the Workload Analysis for Scalable Processors (WASP) Suite, a comprehensive C-based frame-

TABLE IV  
WORKLOAD CLASSIFICATION RESULTS

| Workload | Cores | Intensity | Latency ( $\mu s$ ) | Class                 |
|----------|-------|-----------|---------------------|-----------------------|
| CPU      | 1     | 25%       | 0.057               | Compute-Path-Dominant |
| CPU      | 1     | 75%       | 0.058               | Compute-Path-Dominant |
| Memory   | 1     | 25%       | 0.016               | Memory-Path-Dominant  |
| Memory   | 1     | 75%       | 0.017               | Memory-Path-Dominant  |
| I/O      | 1     | 25%       | 0.000               | Storage-Path-Dominant |
| I/O      | 1     | 75%       | 0.000               | Storage-Path-Dominant |
| Mixed    | 1     | 25%       | 0.019               | Composite-Path        |
| Mixed    | 1     | 75%       | 0.023               | Composite-Path        |

work for multicore scheduling and microarchitecture performance analysis. Key findings are:

- Under normalized operation accounting, CPU has the largest mean per-operation latency in the frozen snapshot (about 0.057  $\mu s$ ), while memory and mixed are lower.
- I/O latency in this normalization is below 0.001  $\mu s$  at the reported precision, and therefore appears as 0.000 in tables/figures.
- The current results set is single-core; multicore scaling and scheduling conclusions are deferred until the full core-count sweep is complete.
- Tail latency analysis (P50/P95/P99) reveals measurable runtime variability across all workloads, reinforcing the need for percentile-based reporting beyond mean-only metrics.

Future work will extend WASP with NUMA-aware scheduling analysis, real-time hardware counter integration using eBPF probes, and support for heterogeneous core (P-core/E-core) architectures.

## REFERENCES

- [1] Standard Performance Evaluation Corporation, “SPEC CPU 2017 Benchmark Suite,” 2017. [Online]. Available: <https://www.spec.org/cpu2017/>
- [2] C. Bienia, “Benchmarking Modern Multiprocessors,” Ph.D. dissertation, Princeton University, 2011.
- [3] L. McVoy and C. Staelin, “LMbench: Portable Tools for Performance Analysis,” in *Proc. USENIX Annual Technical Conf.*, 1996.
- [4] J. Lozi *et al.*, “The Linux scheduler: A decade of wasted cores,” in *Proc. EuroSys*, 2016.
- [5] V. Weaver *et al.*, “Measuring the Overhead of Hardware Performance Counter Infrastructure,” in *IEEE ISPASS*, 2013.
- [6] Brendan Gregg, “perf-tools,” GitHub. [Online]. Available: <https://github.com/brendangregg/perf-tools>
- [7] Michael Larabel, “Phoronix Test Suite,” 2023. [Online]. Available: <https://www.phoronix-test-suite.com/>